Heaven's light is our guide

Rajshahi University of Engineering & Technology

Department of Electrical & Computer Engineering

Course No. : ECE 2112

Course Title : Digital Techniques Sessional

Lab Report : 01

Submitted by:	**Submitted to:**
Rafin Mahfuz Roll : 2410029 Department of Electrical & Computer Engineering, RUET	Mst. Mazeda Noor Tasnim Lecturer Department of Electrical & Computer Engineering, RUET

Date of Submission : 26 July 2026

Introduction:

Digital logic circuits are the foundation of modern electronic systems, including computers, calculators, communication devices, and embedded systems. Logic gates perform basic Boolean operations on binary inputs (0 and 1) to produce a desired output. Among all logic gates, the **NAND** and **NOR** gates are known as **universal gates** because they can be used to implement any other logic gate or digital circuit.

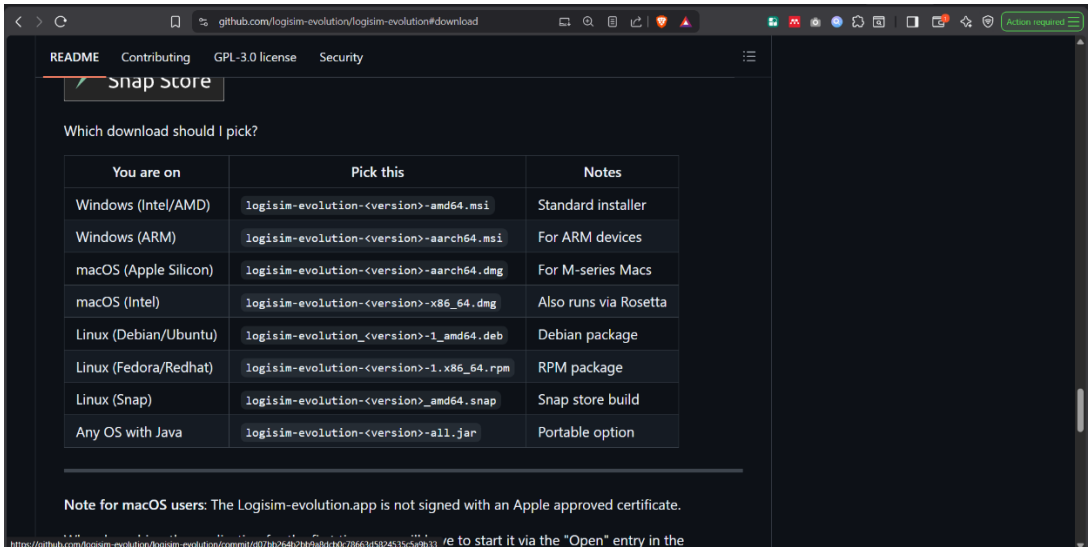
In this experiment, the basic logic gates **AND**, **OR**, and **NOT** are implemented using only **NAND** and **NOR** gates. This demonstrates the universality of these gates and reinforces the concepts of Boolean algebra and logic gate design. Furthermore, a **Full Adder** circuit is constructed using logic gates to perform binary addition of three input bits, producing a **Sum** and a **Carry** output. The experiment also includes the implementation of a **Binary-to-BCD converter**, which converts a binary number into its equivalent Binary-Coded Decimal (BCD) representation for digital display and processing applications.

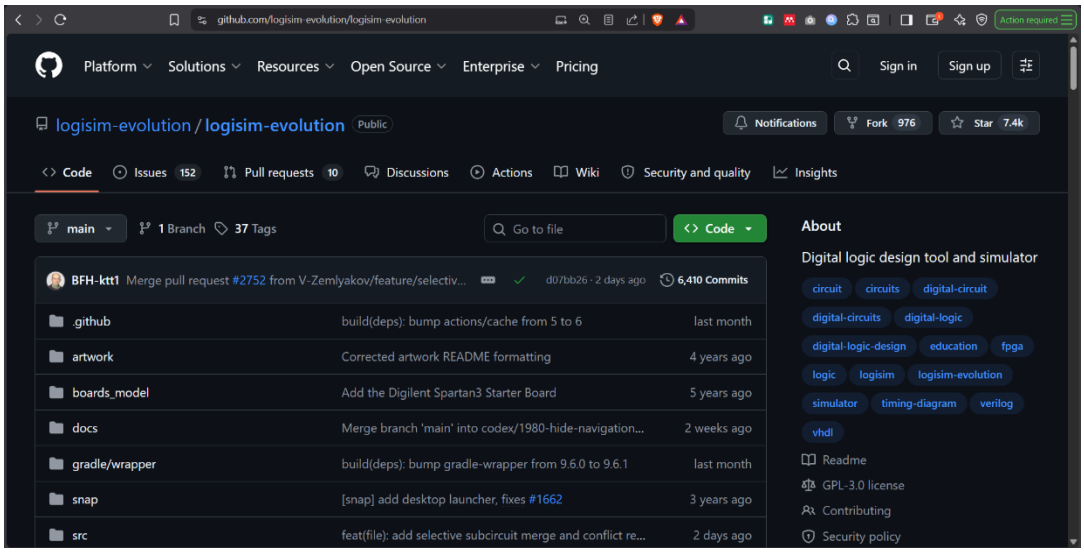
The objective of this experiment is to understand the practical implementation of combinational logic circuits using universal gates, verify their truth tables, and gain hands-on experience in designing digital systems. This knowledge forms the basis for the design and analysis of more complex digital circuits used in modern electronic and computer engineering applications.

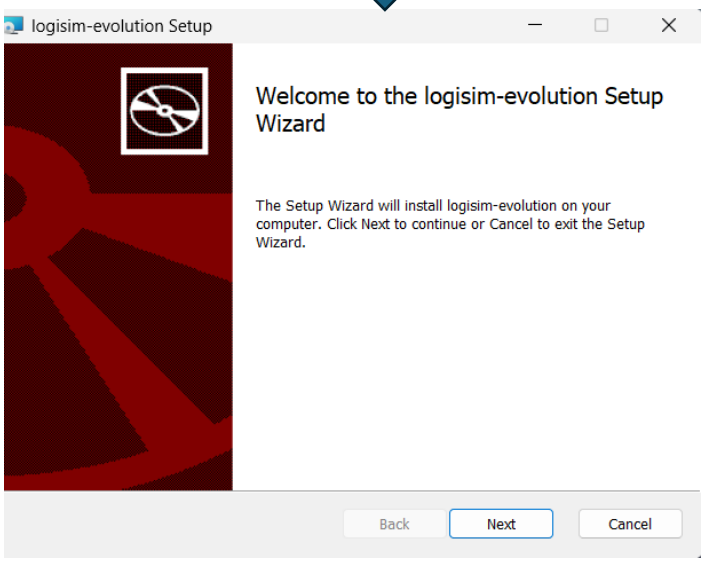
Installation of Logisim Evolution:

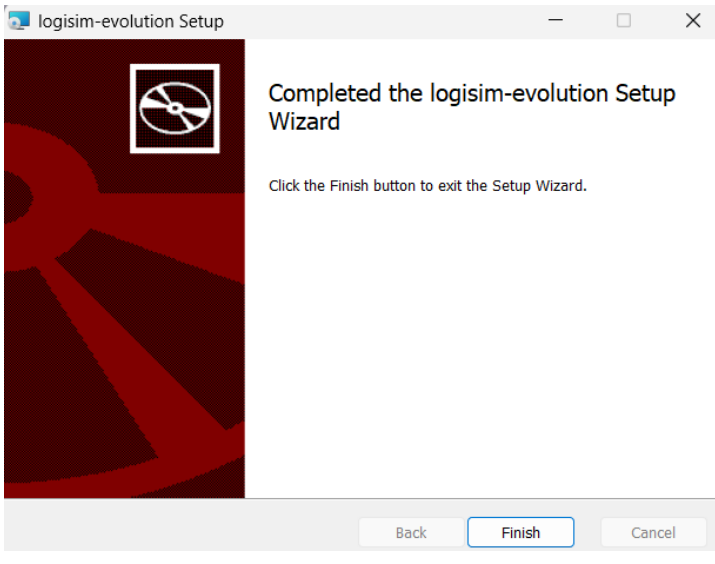
Procedure for Downloading and Installing Logisim Evolution

- Opened a web browser and visited the official Logisim Evolution GitHub repository.
- Navigated to the Releases section of the repository.
- Downloaded the latest Logisim Evolution release suitable for Windows.
- Extracted the downloaded file (if it was in ZIP format).
- Installed Java (if required) to run the application.
- Launched the Logisim Evolution application by opening the executable file or running the JAR file.
- Verified that the software opened successfully and was ready for designing and simulating digital logic circuits.
- Used Logisim Evolution to implement and verify the AND, OR, and NOT gates using NAND and NOR gates, design a Full Adder, and implement a Binary-to-BCD converter.


github.com/logisim-evolution/logisim-evolution#download
README
Contributing
GPL-3.0 license
Security
Which download should I pick?
You are on
Pick this
Notes
Windows (Intel/AMD)
logisim-evolution-<version>-amd64.msi
Standard installer
Windows (ARM)
logisim-evolution-<version>-aarch64.msi
For ARM devices
macOS (Apple Silicon)
logisim-evolution-<version>-aarch64.dmg
For M-series Macs
macOS (Intel)
logisim-evolution-<version>-x86_64.dmg
Also runs via Rosetta
Linux (Debian/Ubuntu)
logisim-evolution_<version>-1_amd64.deb
Debian package
Linux (Fedora/Redhat)
logisim-evolution-<version>-1.x86_64.rpm
RPM package
Linux (Snap)
logisim-evolution-<version>_amd64.snap
Snap store build
Any OS with Java
logisim-evolution-<version>-all.jar
Portable option
Note for macOS users: The Logisim-evolution.app is not signed with an Apple approved certificate.


github.com/logisim-evolution/logisim-evolution
Platform
Solutions
Resources
Open Source
Enterprise
Pricing
Sign in
Sign up
logisim-evolution / logisim-evolution
Public
Notifications
Fork 976
Star 7.4k
Code
Issues 152
Pull requests 10
Discussions
Actions
Wiki
Security and quality
Insights
main
1 Branch
37 Tags
Go to file
Code
About
Digital logic design tool and simulator
BFH-ktt1 Merge pull request #2752 from V-Zemlyakov/feature/selectiv...
d07bb26 · 2 days ago
6,410 Commits
.github
build(deps): bump actions/cache from 5 to 6
last month
artwork
Corrected artwork README formatting
4 years ago
boards_model
Add the Digilent Spartan3 Starter Board
5 years ago
docs
Merge branch 'main' into codex/1980-hide-navigation...
2 weeks ago
gradle/wrapper
build(deps): bump gradle-wrapper from 9.6.0 to 9.6.1
last month
snap
[snap] add desktop launcher, fixes #1662
3 years ago
src
feat(file): add selective subcircuit merge and conflict re...
2 days ago
Readme
GPL-3.0 license
Contributing
Security policy


logisim-evolution Setup
Welcome to the logisim-evolution Setup Wizard
The Setup Wizard will install logisim-evolution on your computer. Click Next to continue or Cancel to exit the Setup Wizard.
Back
Next
Cancel


logisim-evolution Setup
Completed the logisim-evolution Setup Wizard
Click the Finish button to exit the Setup Wizard.
Back
Finish
Cancel

Experiment 1: Implementation of AND Gate Using NAND Gate

Objective: To implement an AND gate using only NAND gates and verify its truth table.

Theory: The AND gate is one of the fundamental logic gates in digital electronics. It performs the logical multiplication of two binary inputs. The output of an AND gate is HIGH (1) only when both inputs are HIGH (1). For all other input combinations, the output remains LOW (0).

Boolean Expression of AND Gate

$$Y = A \cdot B$$

where,

- A = First input
- B = Second input
- Y = Output

Circuit Diagram:

Truth Table:

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Discussion:

This experiment demonstrates that an AND gate can be implemented using only NAND gates, proving the universality of the NAND gate.

Experiment 2:

Implementation of OR Gate Using NAND Gate

Objective:

To implement an OR gate using only NAND gates and verify its truth table.

Theory: The OR gate is a basic digital logic gate that produces a logic HIGH (1) output whenever at least one of its inputs is HIGH (1). It produces a LOW (0) output only when both inputs are LOW (0).

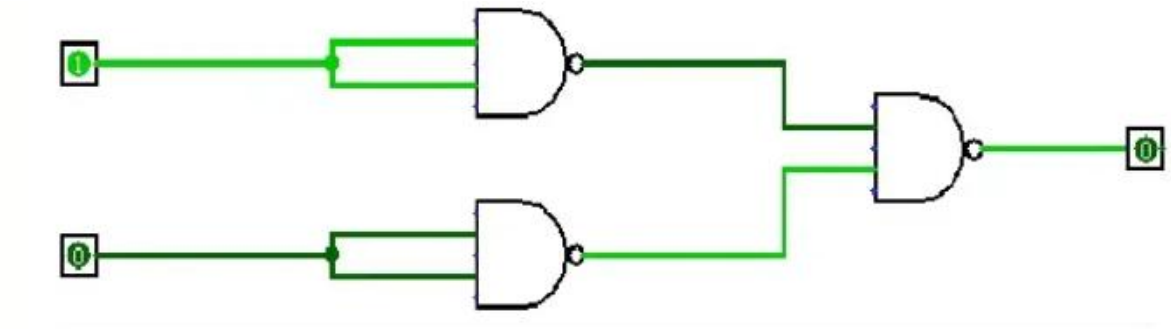
Boolean Expression of OR Gate:

$$\boldsymbol{Y = A + B}$$

Where,

- A = First input
- B = Second input
- Y = Output

Circuit Diagram:



Truth Table:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Discussion:

The output obtained from the NAND gate implementation exactly matched the output of the standard OR gate for all four input combinations. The circuit correctly produced a HIGH output whenever at least one input was HIGH and produced a LOW output only when both inputs were LOW.

Experiment 3:

Implementation of NOT Gate Using NAND Gate

Objective:

To implement an NOT gate using only NAND gates and verify its truth table.

Theory:

The NOT gate, also known as an inverter, is a basic logic gate that produces the complement of its input. It has one input and one output. If the input is logic 1, the output becomes logic 0, and if the input is logic 0, the output becomes logic 1.

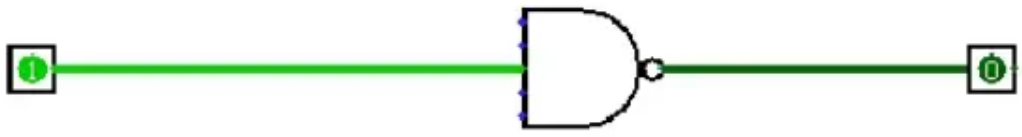
Boolean Expression of NOT Gate

$$Y = \bar{A}$$

where,

- A = Input
- Y = Output
-

Circuit Diagram:



Truth Table:

A	Y
0	1
1	0

Discussion:

The implemented circuit produced the complement of the input for both possible input values. When the input was LOW (0), the output became HIGH (1). When the input was HIGH (1), the output became LOW (0). The observed outputs matched the truth table of a standard NOT gate.

Experiment 4:

Implementation of AND Gate Using NOR Gates

Objective:

To implement an AND gate using only NOR gates and verify its truth table.

Theory: The AND gate is a fundamental logic gate that performs the logical multiplication of two binary inputs. It produces a logic HIGH (1) output only when both inputs are HIGH (1). If either input is LOW (0), the output becomes LOW (0).

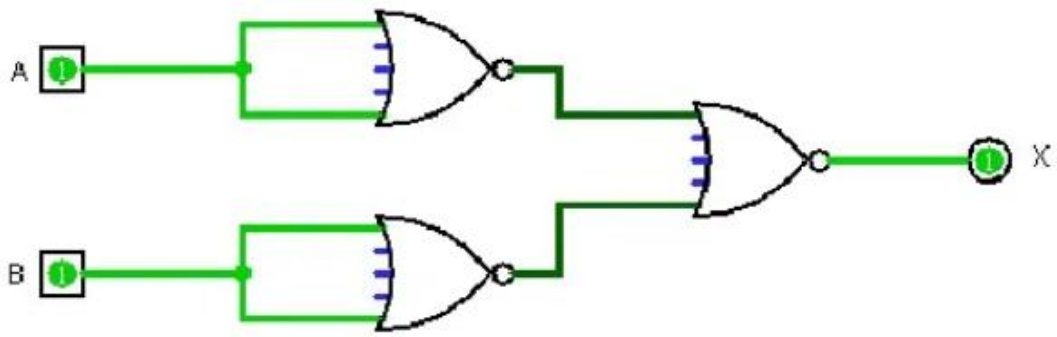
Boolean Expression of AND Gate

$$Y = A \cdot B$$

where,

- **A** = First input
- **B** = Second input
- **Y** = Output

Circuit Diagram:



Truth Table:

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Discussion:

The experiment successfully demonstrated the implementation of an AND gate using only NOR gates.

Experiment 5:

Implementation of OR Gate Using NOR Gates

Objective:

To implement an OR gate using only NOR gates and verify its truth table.

Theory: The OR gate is one of the basic logic gates in digital electronics. It performs the logical addition of two binary inputs. The output of an OR gate is HIGH (1) when at least one of the inputs is HIGH (1). It produces a LOW (0) output only when both inputs are LOW (0).

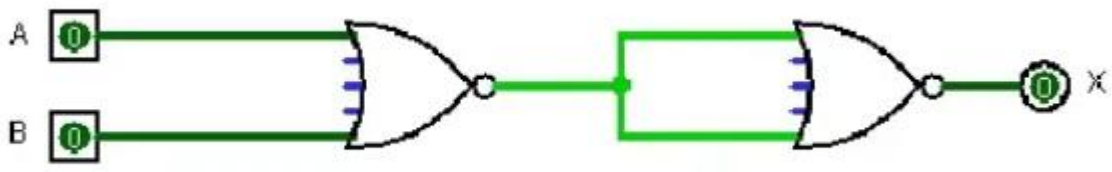
Boolean Expression of OR Gate

$$Y = A + B$$

where,

- **A** = First input
- **B** = Second input
- **Y** = Output

Circuit Diagram:



Truth Table:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Discussion:

The experiment successfully demonstrated the implementation of an OR gate using only NOR gates.

Experiment 6:

Implementation of NOT Gate Using NOR Gates

Objective:

To implement an NOT gate using only NOR gates and verify its truth table.

Theory: The NOT gate, also known as an inverter, is one of the fundamental logic gates in digital electronics. It has one input and one output. The output is always the logical complement of the input. If the input is HIGH (1), the output becomes LOW (0), and if the input is LOW (0), the output becomes HIGH (1).

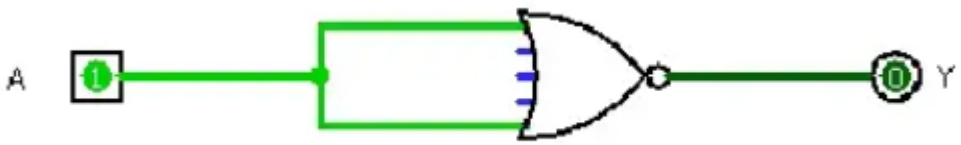
Boolean Expression of NOT Gate

$$Y = \bar{A}$$

where,

- **A** = Input
- **Y** = Output

Circuit Diagram:



Truth Table:

A	Y
0	1
1	0

Discussion:

The experiment successfully demonstrated the implementation of an NOT gate using only NOR gates.

Experiment 7:

Implementation of a Full Adder Using Logic Gates

Objective:

To design and implement a Full Adder circuit using logic gates in Logisim Evolution.

Theory:

A Full Adder is a combinational logic circuit that performs the addition of three one-bit binary inputs: two significant bits (A and B) and an input carry (Cin). It produces two outputs:

- Sum (S): Represents the least significant bit (LSB) of the addition.
- Carry-out (Cout): Represents the carry generated from the addition and is passed to the next higher bit.

Unlike a Half Adder, which adds only two input bits, a Full Adder includes a carry input, making it suitable for multi-bit binary addition in digital systems.

The Full Adder can be implemented using two Half Adders and one OR gate or directly using XOR, AND, and OR gates.

Boolean Expressions

Sum

$$S = A \oplus B \oplus C\,in$$

Carry-out

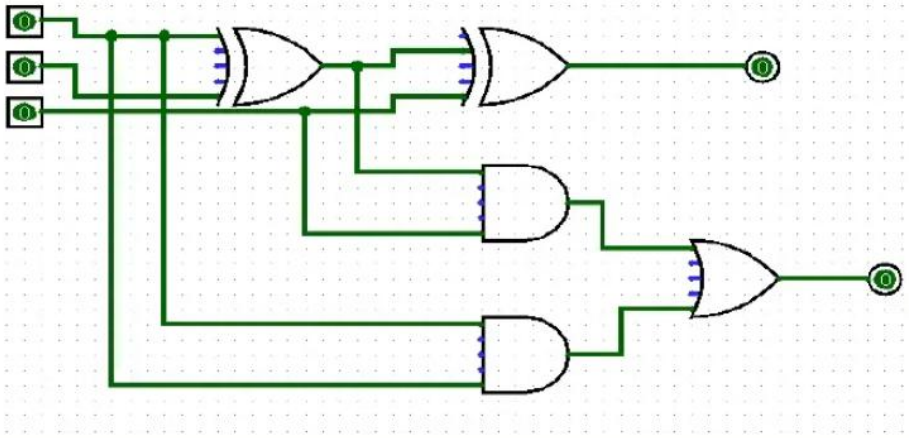
$$C_{out} = AB + AC_{in} + BC_{in}$$

An equivalent expression commonly used in circuit design is:

$$C_{out} = AB + Cin(A \oplus B)$$

The Sum output becomes 1 when an odd number of inputs are 1, while the Carry-out becomes 1 whenever two or more inputs are 1.

Circuit Diagram



Truth Table:

A	B	Cin	Sum (S)	Carry (Cout)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Discussion:

The experiment successfully demonstrated the implementation of a Full Adder using logic gates. The circuit correctly added three one-bit binary inputs and generated accurate **Sum** and **Carry-out** outputs. The simulation results matched the theoretical truth table, confirming the correctness of the circuit design. This experiment illustrates the importance of combinational logic circuits in performing binary arithmetic operations.

Experiment 8:

Implementation of a Binary-to-BCD Converter

Objective:

To design and implement a Binary-to-BCD (Binary-Coded Decimal) converter using Logisim Evolution.

Theory:

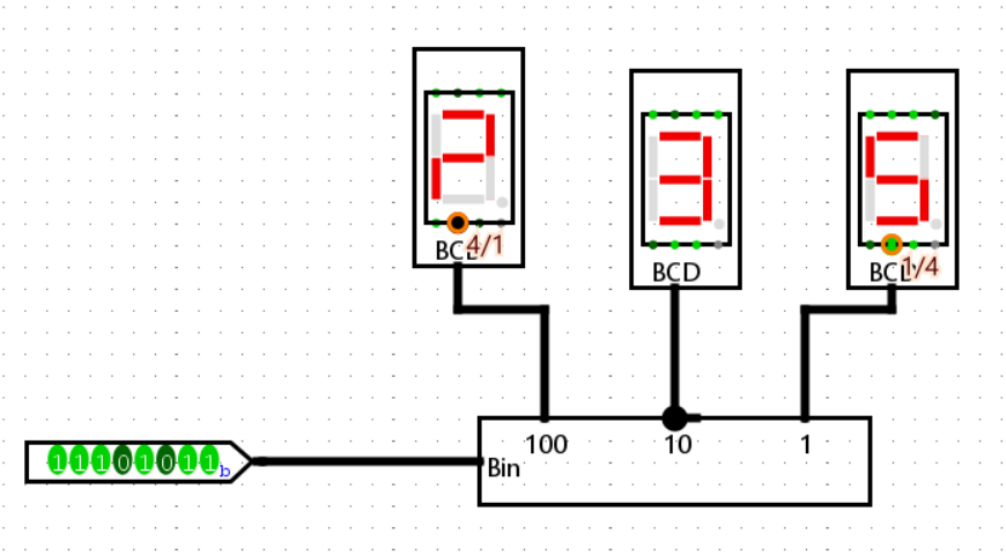
A Binary-to-BCD converter is a combinational logic circuit that converts a binary number into its equivalent Binary-Coded Decimal (BCD) representation. In the BCD system, each decimal digit is represented separately using a 4-bit binary code.

For example,

- Decimal **0** → **0000**
- Decimal **5** → **0101**
- Decimal **9** → **1001**

When binary numbers greater than 1001_2 (9_{10}) are represented, they require two BCD digits: one for the tens place and one for the ones place.

Circuit Diagram:



Truth Table:

Binary	Decimal	BCD Output
1010	10	0001 0000
1011	11	0001 0001
1100	12	0001 0010
1111	15	0001 0101

Discussion:

The experiment successfully demonstrated the implementation of a Binary-to-BCD converter using Logisim Evolution. The converter accurately transformed 4-bit binary numbers into their corresponding BCD representations. The simulation results matched the theoretical conversion table, confirming the correctness of the circuit. Binary-to-BCD conversion is an essential process in digital electronics because most display devices, such as seven-segment displays, operate using BCD inputs rather than pure binary values. This experiment highlights the practical importance of data conversion circuits in digital systems, including calculators, digital clocks, embedded controllers, and measurement instruments.

Conclusion:

The experiments were successfully completed using Logisim Evolution. Basic logic gates (AND, OR, and NOT) were implemented using NAND and NOR universal gates, and their outputs matched the theoretical truth tables. The Full Adder and Binary-to-BCD Converter were also designed and verified successfully. These experiments improved the understanding of Boolean algebra, combinational logic circuits, and the practical applications of digital electronics in modern computing and embedded systems.